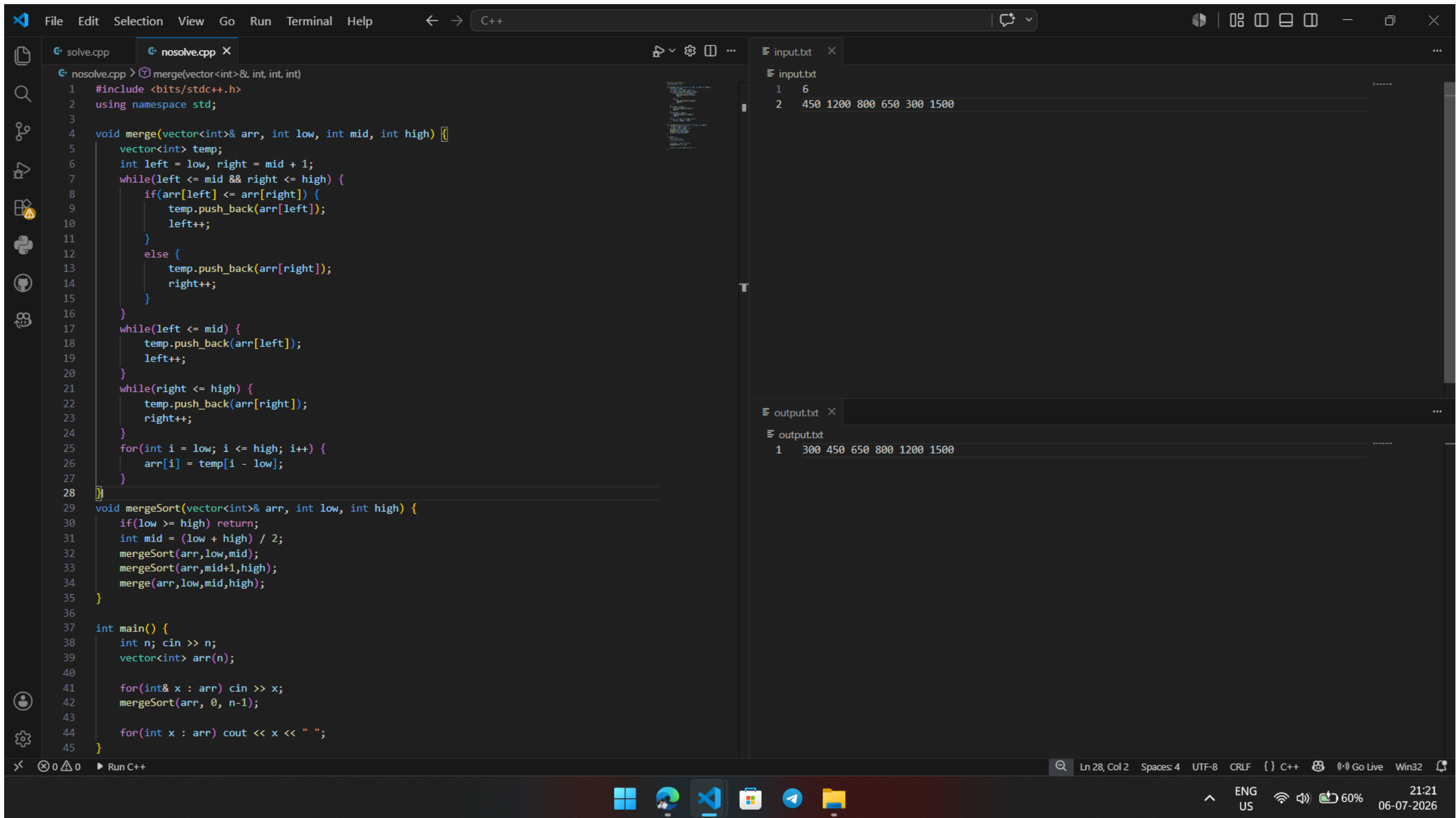
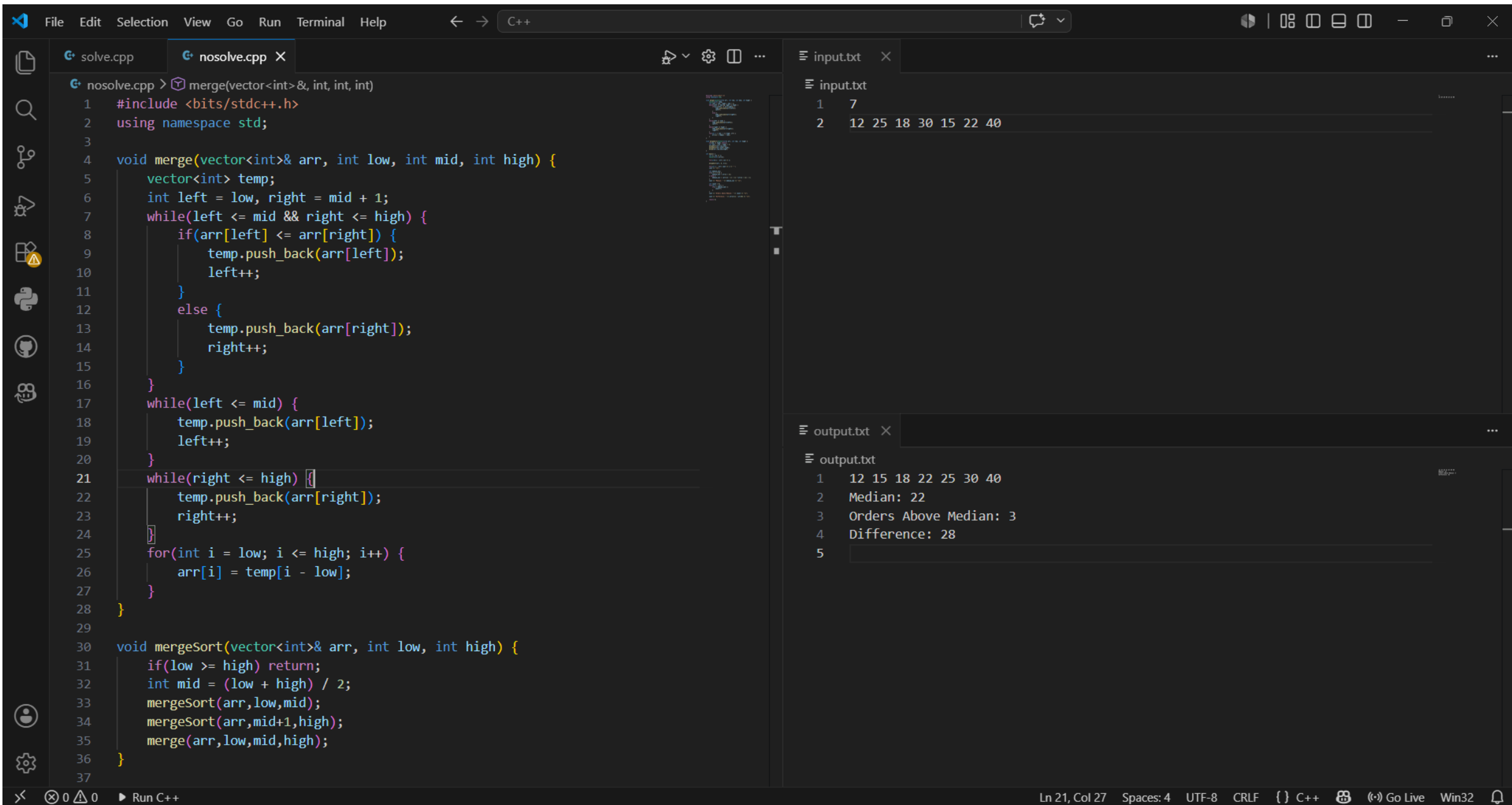
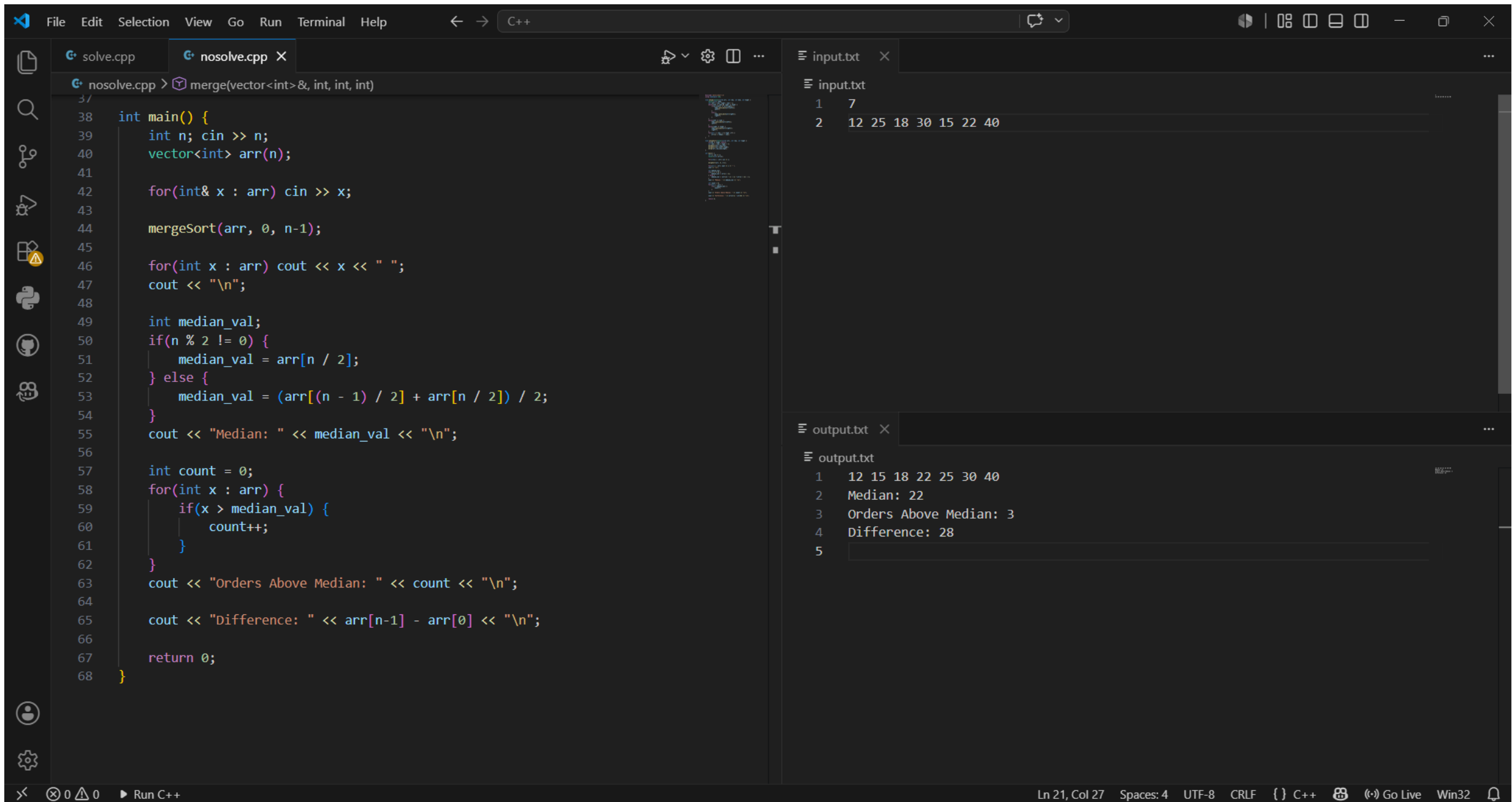




LOGIC: Merge Sort

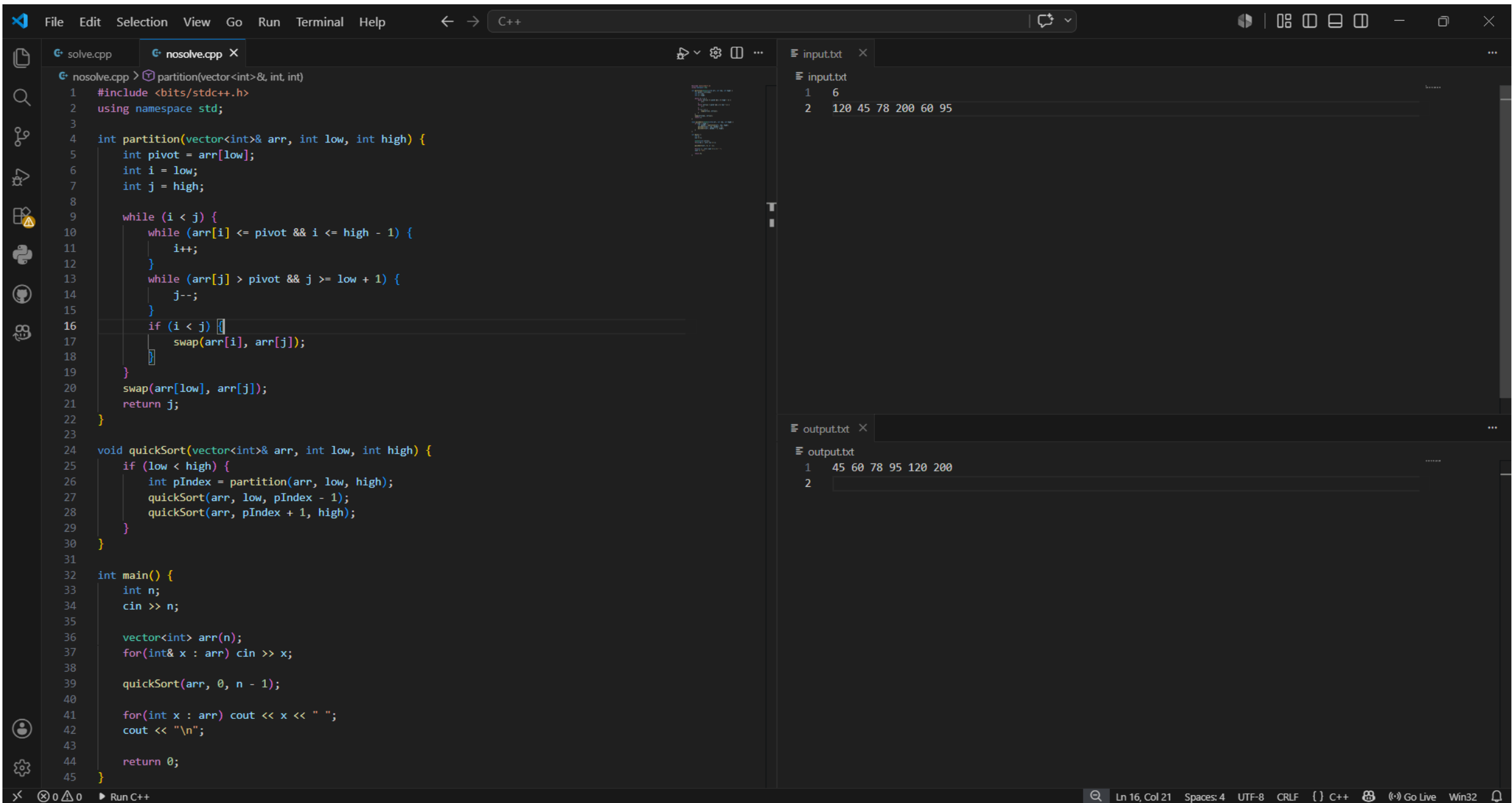
1. We use a divide and conquer approach, so we break the main array into two halves using $\text{mid} = (\text{low} + \text{high}) / 2$ until we hit the base case where $\text{low} \geq \text{high}$.
2. we recursively call the function for both the left half and right half to sort them individually.
3. after breaking it down to single elements, we call the merge function to start joining them back together in ascending order.
4. inside the merge step, we use two pointers left and right to compare the elements and push the smaller one into our `vector<int> temp`.
5. if any elements are left over in either half, we just push them directly into the temp array.
6. finally, we copy all the sorted elements from temp back into it's correct original positions in arr.
7. Since we divide by half and merge n elements, it gives us a very efficient time complexity for large datasets.
($N \log_2 N$)





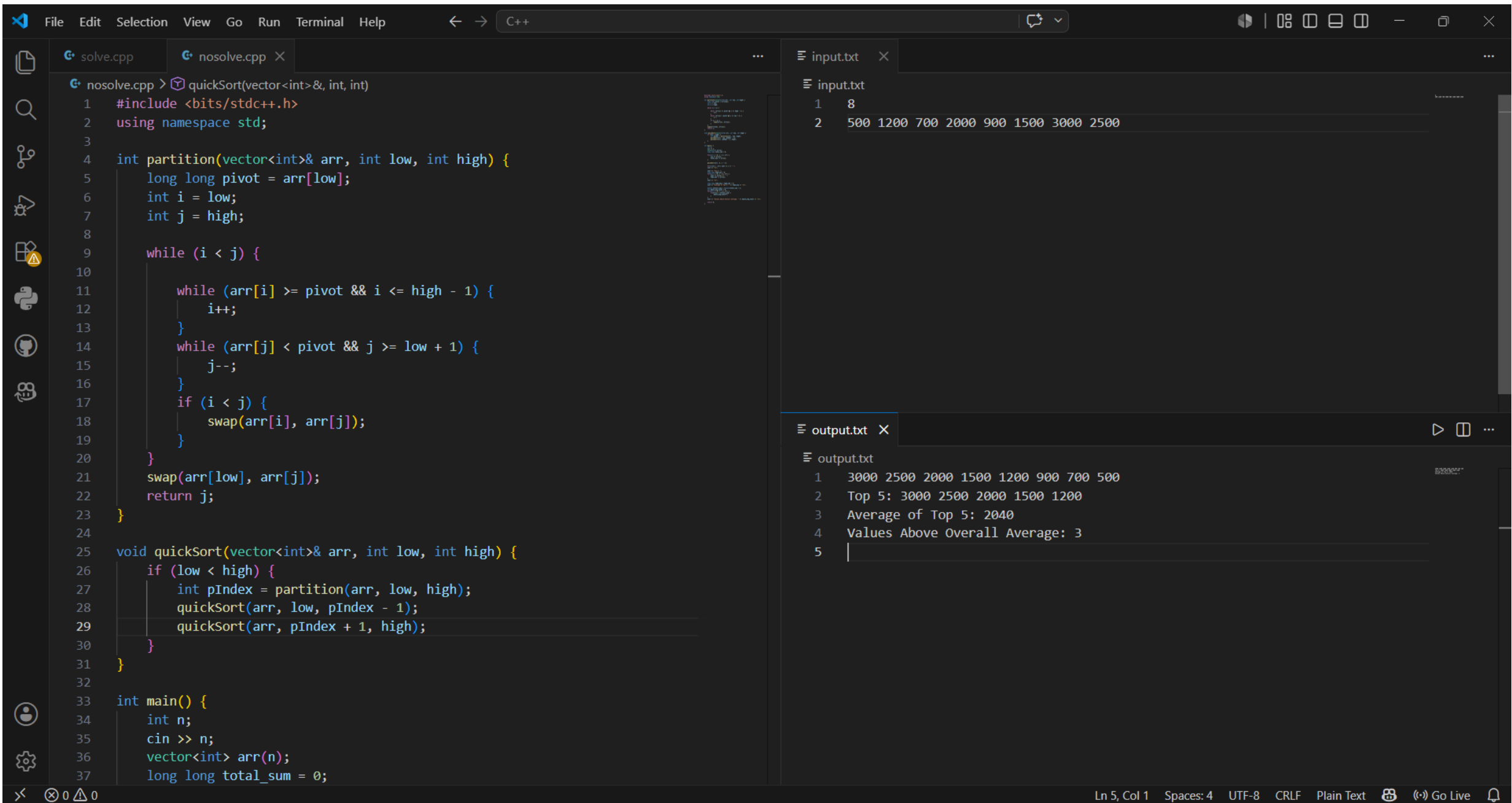
LOGIC: E-commerce Order (Merge Sort)

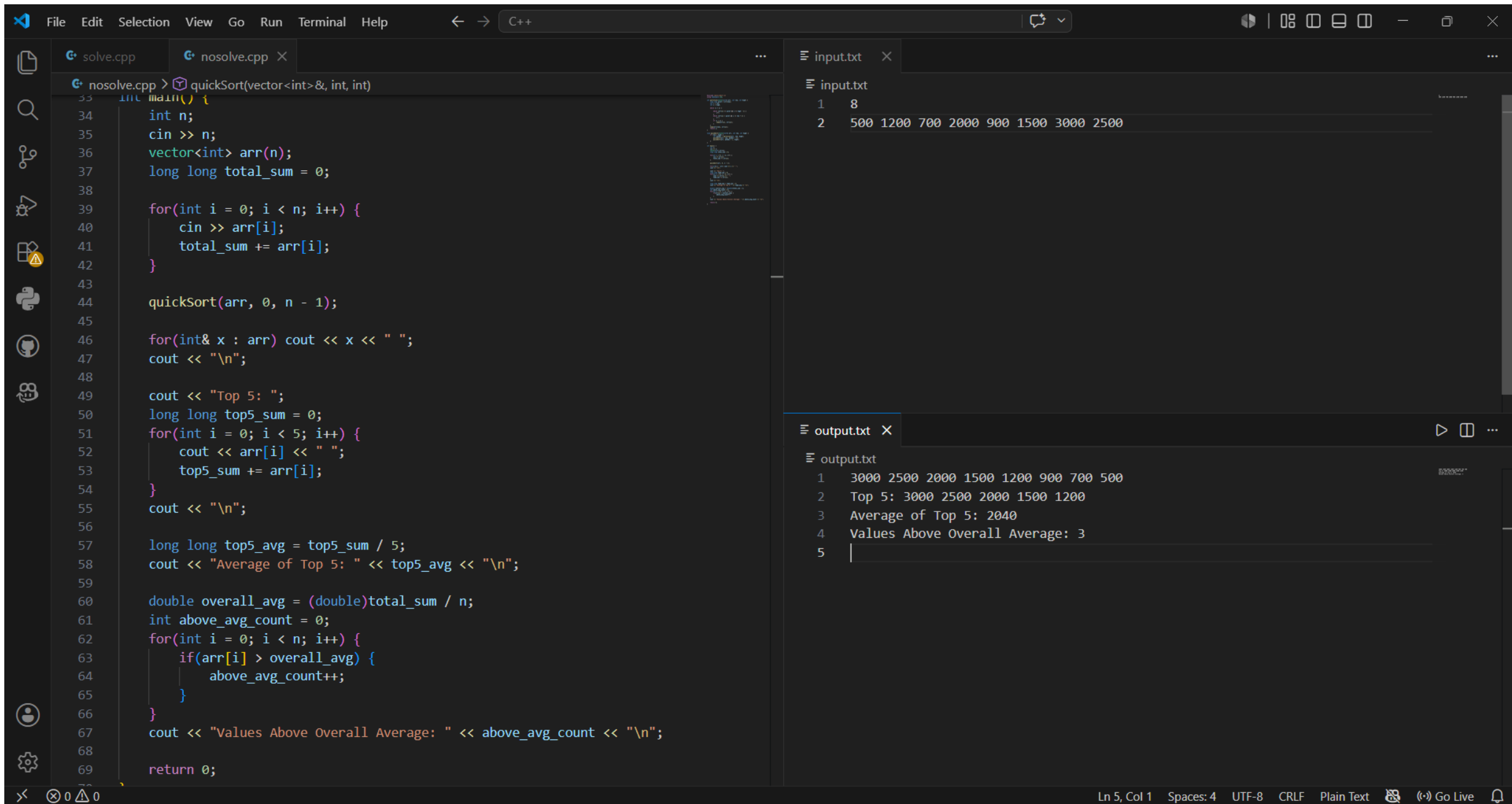
1. We will take the relevant inputs and use the exact same merge sort function to sort the array.
2. we know merge sort breaks the dataset down and merges it back in ascending order so every element takes it's correct place.
3. after the sorting is done, we print the array using a simple for loop.
4. for the median, we check if n is even or odd. If it's odd, the median is just the middle element $\text{arr}[n / 2]$. if even, we just average the two middle elements.
5. we traverse through all the elements again and count how many orders have a time strictly greater than our median_val.
6. finally, since the array is already sorted, the fastest time is at $\text{arr}[0]$ and slowest is at $\text{arr}[n-1]$. we just subtract them to get the difference.
7. This gives us the analytical report without messing up the better time complexity. ($N \log_2 N$)



LOGIC: Server Response Time (Quick Sort)

1. We pick a pivot element, let's assume the first element of our array to be the pivot.
2. we use two pointers i and j , where i starts from the low index and j starts from the high index.
3. i moves forward to find an element strictly greater than the pivot, and j moves backwards to find an element smaller than or equal to the pivot.
4. if i is still less than j , we swap $arr[i]$ and $arr[j]$. this pushes the smaller elements to the left side and larger ones to the right side of the array.
5. after i and j cross each other, it is 100% sure that the j 'th index is the correct place for our pivot. so, we swap the pivot with the j 'th element.
6. now the pivot takes it's correct place, so we just recursively call the quick sort function for the left sub-array and right sub-array.
7. This is an efficient in-place sorting technique as it doesn't need a temporary array like merge sort. ($N \log_2 N$)





quick_sort_medium.txt

File Edit View

H1 H2 H3 H4 H5 H6 H7 H8 H9 H10 H11 H12 H13 H14 H15 H16 H17 H18 H19 H20 H21 H22 H23 H24 H25 H26 H27 H28 H29 H30 H31 H32 H33 H34 H35 H36 H37 H38 H39 H40 H41 H42 H43 H44 H45 H46 H47 H48 H49 H50 H51 H52 H53 H54 H55 H56 H57 H58 H59 H60 H61 H62 H63 H64 H65 H66 H67 H68 H69 H70 H71 H72 H73 H74 H75 H76 H77 H78 H79 H80 H81 H82 H83 H84 H85 H86 H87 H88 H89 H90 H91 H92 H93 H94 H95 H96 H97 H98 H99 H100

LOGIC: Financial Trade Values (Quick Sort)

#: small correction in the pdf, the values above overall average are 4, not 3.

1. We will take the relevant inputs and calculate the total sum of all trades as we take the input.
2. since the question asks for descending order, we pick the first element to be the pivot.
3. we use two pointers i and j, starting from the low and high indices.
4. i moves forward to find an element strictly smaller than the pivot, and j moves backwards to find an element greater than or equal to the pivot.
5. if i is still less than j, we swap arr[i] and arr[j]. this pushes the larger elements to the left side and smaller ones to the right side of the array.
6. after they cross, it is 100% sure that the j'th index is the correct place for our pivot, so we swap the pivot with the j'th element.
7. now the pivot takes it's correct place, and we recursively call quick sort for both halves.
8. after the array is sorted, we just print the top 5, calculate their average, and use our total sum to find the overall average so we can count how many trades were greater than it.
9. This gives the analytics team exactly what they need without using extra memory. ($N \log_2 N$)

Ln 3, Col 72 | 1,213 characters | Plain text | 70% | Windows (CRLF) | UTF-8

FileEditSelectionViewGoRunTerminalHelp

C++

input.txt

input.txt

output.txt

nosolve.cpp

main()

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  void heapify(vector<int>& arr, int n, int i) {
4      int largest = i;
5      int left = 2 * i + 1;
6      int right = 2 * i + 2;
7
8      if (left < n && arr[left] > arr[largest]) {
9          largest = left;
10     }
11
12     if (right < n && arr[right] > arr[largest]) {
13         largest = right;
14     }
15
16     if (largest != i) {
17         swap(arr[i], arr[largest]);
18         heapify(arr, n, largest);
19     }
20 }
21 void heapSort(vector<int>& arr, int n) {
22     for (int i = n / 2 - 1; i >= 0; i--) {
23         heapify(arr, n, i);
24     }
25
26     for (int i = n - 1; i >= 0; i--) {
27         swap(arr[0], arr[i]);
28         heapify(arr, i, 0);
29     }
30 }
31
32 int main() {
33     int n;
34     cin >> n;
35     vector<int> arr(n);
36     for(int& x : arr) cin >> x;
37     heapSort(arr, n);
38
39     for(int i = 0; i < n; i++) {
40         cout << arr[i] << (i == n - 1 ? "" : " ");
41     }
42     cout << "\n";
43
44     return 0;
45 }
```

input.txt

input.txt

output.txt

output.txt

output.txt

Ln 45, Col 2

Spaces: 4

UTF-8

CRLF

{ }

C++

Go Live

Win32

heap_sort_easy.txt

File Edit View

H1 **B** *I*

LOGIC: Gaming Tournament Scores (Heap Sort)

1. we first need to build a max heap from the given array. we start from the last non-leaf node, which is $(n/2) - 1$, and go down to 0, calling our heapify function.

2. inside heapify, we assume the current root node i to be the largest. we check it's left child at $2*i + 1$ and right child at $2*i + 2$.

3. if any child is greater than our current largest, we update the largest index.

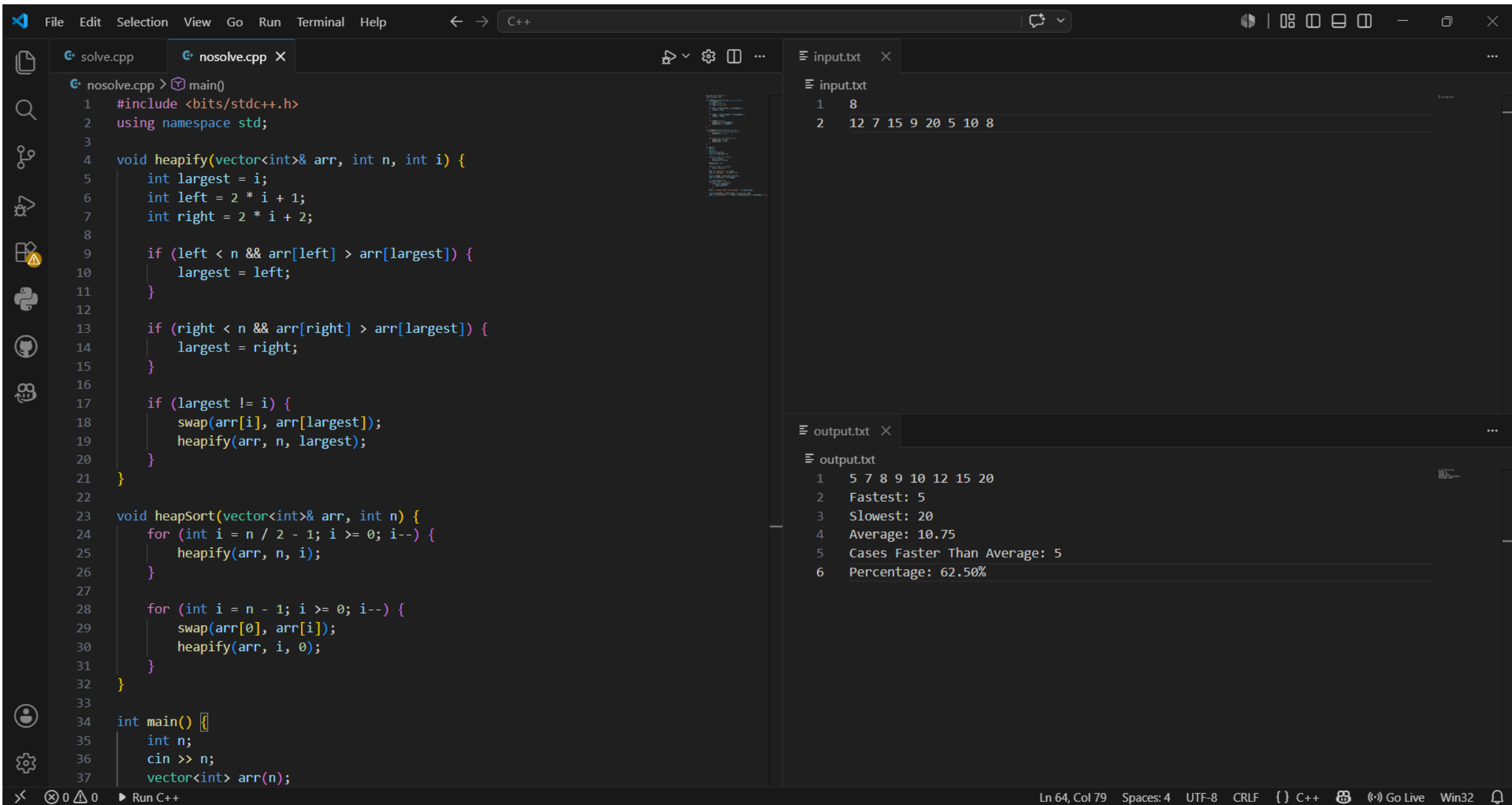
4. if the largest index is not i anymore, we swap them and recursively call heapify on the affected sub-tree to make sure it's still a valid max heap.

5. after the initial max heap is built, the largest element is sitting at the root $arr[0]$. we swap it with the last element of the array and trim the array space by 1.

6. we call heapify again on the root 0 to fix the remaining heap. we repeat this swapping and heapifying process till the whole array is sorted.

7. this gives us a efficient sorting technique without taking any extra space. ($N\log_2 N$)

Ln 15, Col 91 | 990 characters | Plain text | 70% | Windows (CRLF) | UTF-8



FileEditSelectionViewGoRunTerminalHelp

C++

solve.cpp

nosolve.cpp

input.txt

output.txt

nosolve.cpp

main()

23

void heapSort(vector<int>& arr, int n) {

32

}

33

34

int main() {

35

int n;

36

cin >> n;

37

vector<int> arr(n);

38

long long total_sum = 0;

39

40

for(int i = 0; i < n; i++) {

41

cin >> arr[i];

42

total_sum += arr[i];

43

}

44

heapSort(arr, n);

45

46

for(int i = 0; i < n; i++) {

47

cout << arr[i] << " ";

48

}

49

cout << "\nFastest: " << arr[0];

50

cout << "\nSlowest: " << arr[n - 1];

51

52

double average = total_sum / (double)n;

53

cout << "\nAverage: " << average;

54

55

int count_faster = 0;

56

for(int i = 0; i < n; i++) {

57

if(arr[i] < average) {

58

count_faster++;

59

}

60

}

61

cout << "\nCases Faster Than Average: " << count_faster;

62

63

double percentage = (count_faster / (double)n) * 100;

64

cout << "\nPercentage: " << fixed << setprecision(2) << percentage << "%";

65

}

input.txt

1 8

2 12 7 15 9 20 5 10 8

output.txt

1 5 7 8 9 10 12 15 20

2 Fastest: 5

3 Slowest: 20

4 Average: 10.75

5 Cases Faster Than Average: 5

6 Percentage: 62.50%

Ln 64, Col 79

Spaces: 4

UTF-8

CRLF

{ }

C++

Go Live

Win32

LOGIC: Hospital Response Times (Heap Sort)

1. We will take the relevant inputs and calculate the total sum as we read the array.
2. we first need to build a max heap from the given array. we start from the last non-leaf node, which is $(n/2) - 1$, and go down to 0, calling our heapify function.
3. inside heapify, we assume the current root node i to be the largest. we check it's left child at $2*i + 1$ and right child at $2*i + 2$.
4. if any child is greater than our current largest, we update the largest index.
5. if the largest index is not i anymore, we swap them and recursively call heapify on the affected sub-tree to make sure it's still a valid max heap.
6. after the initial max heap is built, the largest element is sitting at the root $arr[0]$. we repeatedly swap it with the last element of the unsorted part and call heapify on the root 0 to fix the remaining heap.
7. once the array is sorted, we just print it. the fastest time is now sitting at $arr[0]$ and slowest is at $arr[n-1]$.
8. we divide the total sum by n to get the average. then we just iterate to count how many cases are strictly smaller than this average (since smaller means faster).
9. finally, we calculate the percentage and print the statistics. ($N\log_2 N$)